

Complexity

What Is It?

Analyzes how the algorithm's efficiency changes as the input grows , focusing on time & space complexity

Complexity = Resources Required

- Time Complexity: Amount of time (CPU operations) needed.
- space Complexity: Amount of memory needed.

Runtime: How much it takes to run a function as the size of the input grows

Breaking Down the Formal Definition

The definition states:

$$O(g(n)) = \{ f(n) \mid \exists c > 0, \exists n_0 > 0, \forall n \geq n_0 : 0 \leq f(n) \leq cg(n) \}$$

Let's break this down in **simple terms**:

- **f(n)**: The actual function representing the algorithm's running time.
- **g(n)**: A function that serves as an **upper bound** for f(n).
- **c**: A constant that **scales up** g(n) to ensure it covers f(n) .
- **n0**: A starting point beyond which the inequality holds.

$$0 \leq f(n) \leq c * g(n) \text{ for all } n \geq n_0$$

This means:

For large values of n, f(n) does not grow faster than g(n) (up to a constant factor c).

Example 1: Proving $f(n)=3n+2$ is in $O(n)$

We need to show that:

$$3n+2 \leq c*n \text{ for all } n \geq n_0$$

Step 1: Choose c and n0

Let's take **n0=1** and **c=5**

For $n \geq 1$

$$3n+2 \leq 5n$$

Since this inequality holds, we conclude:

$$3n+2 \in O(n)$$

✓ **Meaning:** $3n+2$ does not grow faster than n (ignoring constant factors).

Simplifying Complexity Analysis

When analyzing complexity:

- Ignore lower-degree terms as $n \rightarrow \infty$.
- Drop constant coefficients (e.g., $3 \Rightarrow 1$).

This is called Asymptotic Analysis.

What is a Bound in Time Complexity?

A **bound** means a way to describe how fast the running time of an algorithm grows as the input size n increases.

There are different types of bounds:

- **Upper bound** $\rightarrow$ Big-O (worst-case growth rate)
 - **Lower bound** $\rightarrow$ Big- Ω (best-case growth rate)
 - **Tight bound** $\rightarrow$ Big- Θ (both upper and lower at the same time) may not be applicable for some solutions
-

Loose Boundaries

A **loose bound** is when the complexity description is **not very precise**, but it's still correct.

Example:

Suppose an algorithm's actual time complexity is **$O(n)$** .

- Saying it is **$O(n^2)$** is *correct* (because $O(n) \subseteq O(n^2)$),
- But it's a **loose bound**, since it doesn't reflect the actual growth closely.

Loose bounds can exaggerate the growth rate and make the algorithm look worse than it really is.

Tight Boundaries

A **tight bound** is when the complexity is described as **closely as possible**—neither overestimating nor underestimating.

That's when we use **Big- Θ (Theta)**.

Example:

If an algorithm always runs in **$c \cdot n$ steps** (where c is a constant):

- Upper bound = $O(n)$
- Lower bound = $\Omega(n)$
- Since both are the same $\rightarrow \Theta(n)$

So, $\Theta(n)$ is the **tight bound**.

Why defining Complexities is Important?

1. **Predicts Performance:** Helps compare which algorithm will be **faster** for large inputs.
2. **Avoids Inefficiency:** Choosing a bad algorithm can make programs **slow and unusable**.
3. **Scalability:** Ensures that an algorithm **scales well** as data size increases.
4. **Standard Measurement:** Used in coding interviews and competitive programming to analyze code efficiency.

Common Big O Complexities:

1. **$O(1)$** : means execution time remains the same regardless of the input size

لو حسبت $1000 * 100$ مرة واحدة او عشر مرات كل مره هحتاج نفس الوقت عشان احسبها لانها ارقام ثابتة وهفضل تديني نفس الناتج

Example : $O(1)$ – Constant Time

```
int getFirstElement(int arr[]) {  
    return arr[0]; // Always takes the same time, no matter the size of arr  
}
```

◆ **Performance:** The function runs in **constant time** because it only accesses one element.

2. **$\log(n)$** : when the time it takes to complete **reduces** with each step, usually by **dividing the problem in half** at every iteration.

Why Is $O(\log n)$ Fast?

- Instead of iterating through **all** elements, the algorithm **cuts the problem size in half** each time.
- This makes it **much faster** than $O(n)$, $O(n^2)$, or $O(n^3)$.

- Example: If you have **1,000,000** elements:
 - **$O(n)$** → **1,000,000** steps.
 - **$O(\log n)$** → **only ~20 steps!**

Example 1: Binary Search ($O(\log n)$)

Binary search finds an element in a **sorted** array by repeatedly dividing the array in half.

```
int binarySearch(int arr[], int left, int right, int target) {
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid; // Found the target
        if (arr[mid] < target) left = mid + 1; // Search right half
        else right = mid - 1; // Search left half
    }
    return -1; // Target not found
}
```

◆ Why $O(\log n)$?

- Each step **halves** the search space.
- If there are **n** elements, the number of steps is **$\log_2(n)$** .
- Example:
 - **$n = 16$** → **4 steps.**
 - **$n = 1,000,000$** → **20 steps!**

Example 2: Logarithmic Loop ($O(\log n)$)

```
void logLoop(int n) {
    for (int i = 1; i < n; i *= 2) {
        cout << i << " ";
    }
}
```

◆ Why $O(\log n)$?

- i starts at **1** and **doubles** every step: **1, 2, 4, 8, 16, ...**
- The loop runs **$\log_2(n)$** times.

3. $O(n)$: means an operation's execution time increases proportionally with the input size
>> iterating on array

هيزيد برضو بالتوازي معاه ... يعني calcs بتاعي يزيد أو يكبر عدد الخطوات او وقت ال input كل م ال هيزيد iterating كل م عدد عناصره تزيد او حجمه يكبر ساعتها وقت ال loop over an array مثلا لو ب برضو معاه

Example : $O(n)$ – Linear Time

```
void printArray(int arr[], int n) {  
    for (int i = 0; i < n; i++) {  
  
        cout << arr[i] << " "; // Runs n times  
    }  
}
```

◆ **Performance:** The function runs **proportional** to n , so it's $O(n)$.

4. $O(n \log n)$: **hybrid** complexity that combines $O(n)$ (linear time) with $O(\log n)$ (logarithmic time). It often appears in **sorting algorithms** and **divide & conquer approaches**.

Why $O(n \log n)$?

- $O(\log n)$ comes from **dividing the problem into halves** (like merge sort or quicksort).
- $O(n)$ comes from **processing each element** at least once (like merging or partitioning).
- Together, they form $O(n \log n)$.

Where Does $O(n \log n)$ Appear?

It appears in sorting algorithms that use **divide & conquer**, such as:

Algorithm	Complexity	Explanation
Merge Sort	$O(n \log n)$	Recursively splits array ($\log n$) and merges (n)
QuickSort (Best & Avg.)	$O(n \log n)$	Recursively partitions ($\log n$) and processes (n)
Heap Sort	$O(n \log n)$	Builds a heap (n) and extracts max ($\log n$)

Merge Sort Code ($O(n \log n)$)

```
void merge(vector<int>& arr, int left, int mid, int right) {  
    vector<int> leftPart(arr.begin() + left, arr.begin() + mid + 1);
```

```

vector<int> rightPart(arr.begin() + mid + 1, arr.begin() + right + 1);

int i = 0, j = 0, k = left;
while (i < leftPart.size() && j < rightPart.size()) {
    if (leftPart[i] <= rightPart[j]) arr[k++] = leftPart[i++];
    else arr[k++] = rightPart[j++];
}

while (i < leftPart.size()) arr[k++] = leftPart[i++];
while (j < rightPart.size()) arr[k++] = rightPart[j++];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

◆ **Splitting the array → $\log n$**

◆ **Merging back → $O(n)$**

◆ **Total → $O(n \log n)$**

5. $O(n^2)$: when the time it takes to complete grows portionally to the square of the input size.

How to identify ?

1. there are 2 nested loops iterating over the same data
2. each element is compared with every other element
3. The input size is doubled

How to Optimize $O(n^2)$ Algorithms?

1. **Use better sorting algorithms:** Merge Sort $O(n \log n)$, QuickSort $O(n \log n)$.
2. **Use Hashing or Sets** to reduce unnecessary comparisons.
3. **Break out of loops early** if a condition is met.

Example : $O(n^2)$ – Quadratic Time

```
void printPairs(int arr[], int n) {  
    for (int i = 0; i < n; i++) { // outer loop runs n time  
  
        for (int j = 0; j < n; j++) { //inner loop runs n time  
  
            cout << arr[i] << ", " << arr[j] << endl;  
        }  
    }  
}
```

◆ **Performance:** The **nested loop** means it runs $n \times n = n^2$ times

6. $O(n^3)$: when the time it takes to complete grows portionally to the cubic of the input size.

How to identify

1. **Look for three nested loops** iterating over the same input size n .
2. **Check if each element is being compared multiple times.**
3. **If the input size doubles, the runtime increases by $2^3 = 8$ times.**

How to Optimize $O(n^3)$ Algorithms?

1. **Reduce nested loops** by using better **data structures**.
2. **Use dynamic programming** to store previous calculations.
3. **Use sorting + two-pointer techniques** instead of brute-force triplet searches.

Example 5: Checking All Possible Triplets ($O(n^3)$)

```
void findTriplets(int arr[], int n) {  
  
    for (int i = 0; i < n; i++) { // First element  
  
        for (int j = i + 1; j < n; j++) { // Second element  
  
            for (int k = j + 1; k < n; k++) { // Third element
```

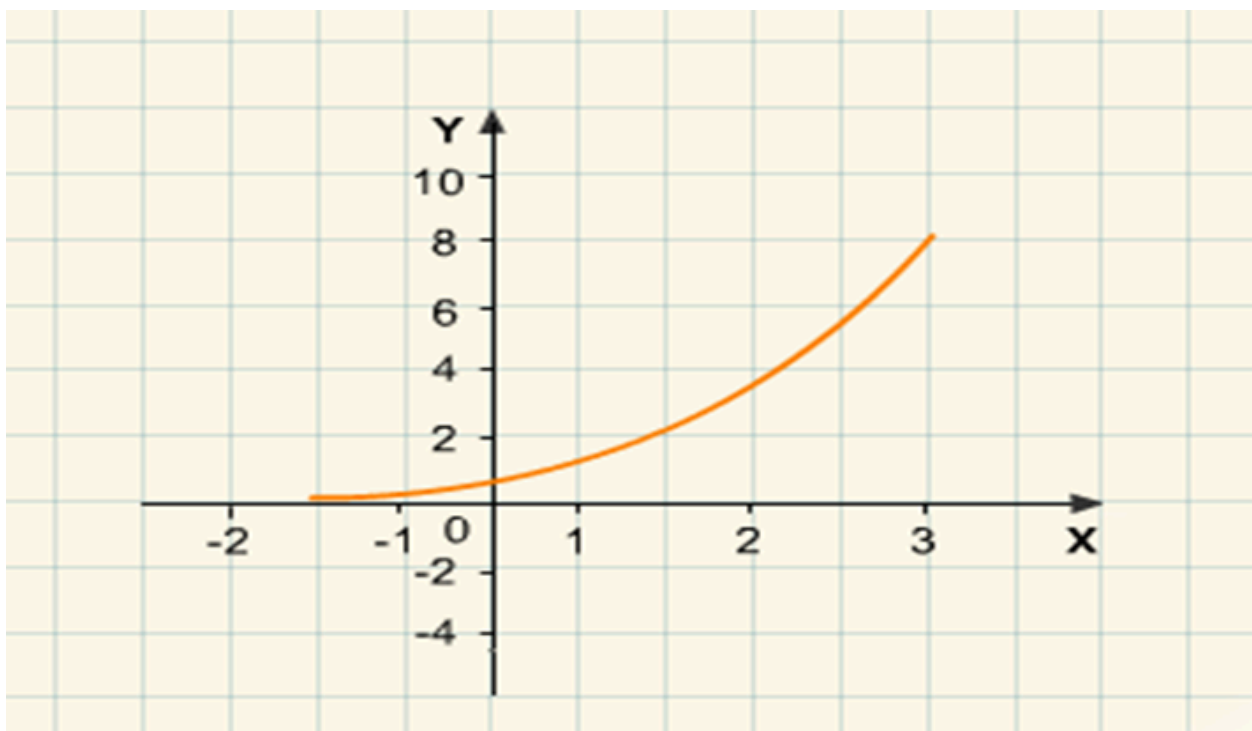
```

cout << arr[i] << ", " << arr[j] << ", " << arr[k] << endl;
}
}
}
}

```

◆ **Use case:** Finding all triplets in an array.

7. $O(2^n)$ "unapplicable" : **exponential time complexity**, where the **runtime doubles** with each increase in input size. This is **one of the worst time complexities**, as it grows extremely fast.



Where Does $O(2^n)$ Appear?

$O(2^n)$ often appears in **recursive algorithms** that generate **all subsets** or **all possible combinations**.

Algorithm	Why $O(2^n)$?
Fibonacci (Naive Recursion)	Calls itself twice per step
Subset Generation	Each element can be included/excluded
Solving NP-Complete Problems (Brute Force)	Tries all possibilities

Example: Fibonacci ($O(2^n)$)

The naive recursive Fibonacci function calls itself **twice** for each number, leading to $O(2^n)$.

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Visualizing $O(2^n)$ Growth

n (Input Size)	$O(2^n)$ Steps (~)
5	32
10	1,024
20	1,048,576
30	1,073,741,824 (1 billion steps!)

$O(2^n)$ explodes very quickly! Anything beyond **30** is often infeasible.

8. **$O(n!)$ "unapplicable"**: **factorial time complexity**, which is the **worst possible growth rate** in common algorithmic problems. It means the **runtime grows extremely fast** as n increases.

Where Does $O(n!)$ Appear?

$O(n!)$ usually appears in problems involving **permutations, backtracking, and brute force searches**.

Algorithm / Problem	Why $O(n!)$?
Generating all permutations	Every element must be placed in every position
Traveling Salesman Problem (TSP) - Brute Force	Tries every possible order of cities
Solving NP-Hard problems with brute force	Explores all possible configurations

Example: Generating All Permutations ($O(n!)$)

Generating all permutations means arranging **n elements** in every possible way.

Since there are **$n!$ ways to arrange n elements**, the complexity is $O(n!)$.

```
void permute(vector<int>& nums, int l, int r) {  
    if (l == r) {
```

```

        for (int num : nums) cout << num << " ";
        cout << endl;
        return;
    }

    for (int i = l; i <= r; i++) {
        swap(nums[l], nums[i]); // Swap elements
        permute(nums, l + 1, r); // Recur
        swap(nums[l], nums[i]); // Backtrack
    }
}

int main() {
    vector<int> nums = {1, 2, 3};
    permute(nums, 0, nums.size() - 1);
}

```

Breakdown of $O(n!)$ Complexity

1. First element has **n** choices.
2. Second element has **$(n-1)$** choices.
3. Third element has **$(n-2)$** choices.
4. **Total combinations** = $n \times (n-1) \times (n-2) \times \dots \times 1 = n!$

For $n = 5$, this means $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ permutations!

For $n = 10$, it becomes 3,628,800! (Too slow!)

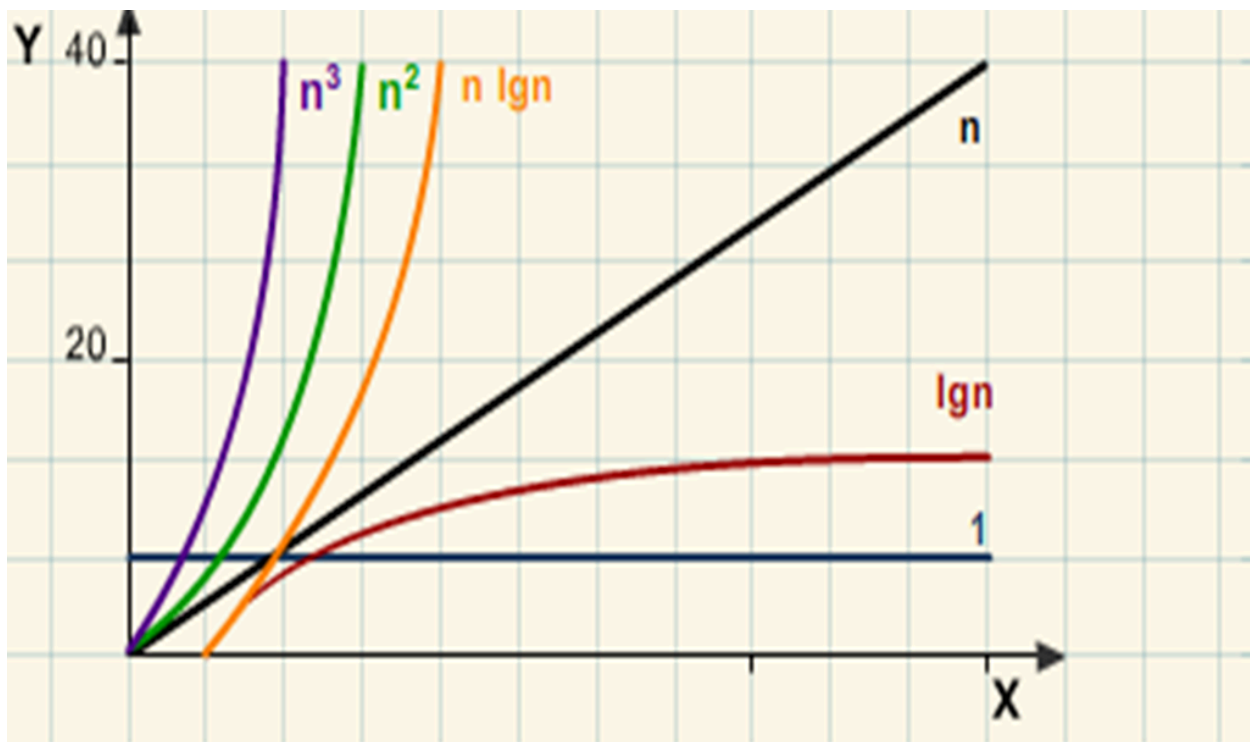
Visualizing $O(n!)$ Growth

n (Input Size)	$O(n!)$ Steps (~)
3	6
5	120
10	3,628,800
20	2.43×10^{18} (Impossible!)

$O(n!)$ grows faster than $O(2^n)$ and is completely impractical for large n !

Common Big O Complexities

Big O Notation	Name	Example	Performance
$O(1)$	Constant Time	Accessing an array element	🔥 Fastest
$O(\log n)$	Logarithmic Time	Binary search	🚀 Very Fast
$O(n)$	Linear Time	Looping through an array	⚡ Good
$O(n \log n)$	Log-Linear Time	Merge sort, Quick sort	⚠️ Okay
$O(n^2)$ $O(n^3)$	Quadratic Time Cubic Time	Nested loops (e.g., Bubble Sort)	🐢 Slow
$O(2^n)$	Exponential Time	Recursive Fibonacci	🛑 Very Slow
$O(n!)$	Factorial Time	Brute-force permutations	❌ Worst



Big-O Rules: Understanding How to Combine Complexities

◆ Rule 1: Addition Rule

$$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$

Used when two algorithms run one after another.

We add their complexities since both contribute to the total runtime.

Example : Reading an array & sorting it

```
void readArray(int x[], int n) {
    for (int i = 0; i < n; i++)
        cin >> x[i]; // Reading n elements
}

void sortArray(int x[], int n) {
    for (int i = 0; i < n; i++) // Outer loop (n iterations)
        for (int j = 0; j < i; j++) // Inner loop (i iterations)
            if (x[j] > x[i])
                swap(x[i], x[j]);
}
```

- **Reading array** $\rightarrow O(n)$
- **Sorting (nested loops)** $\rightarrow O(n^2)$
- **Overall complexity** $\rightarrow O(n + n^2) = O(n^2)$

◆ Rule 2: Multiplication Rule

$O(f(n)) * O(g(n)) = O(f(n) * g(n))$

Used when one function calls another function multiple times.

Total complexity is the product of both complexities.

Example : Reading, Sorting, and Printing M Arrays

```
void readSortWrite(int m, int n) {
    for (int array = 1; array <= m; array++) {
        readArray(x, n); // O(n)
        sortArray(x, n); // O(n^2)
        printArray(x, n); // O(n)
    }
}
```

- **Reading an array** $\rightarrow O(n)$
- **Sorting an array** $\rightarrow O(n^2)$
- **Printing an array** $\rightarrow O(n)$

- **Total per array** $\rightarrow O(n + n^2 + n) = O(n^2)$
 - **Repeating for m arrays** $\rightarrow$ 
 $O(m \times n^2)$
-

How to Determine Big O?

1. **Focus on the dominant term:** If an algorithm has $O(n+n^2)O(n + n^2)O(n+n^2)$, we drop the lower term $\rightarrow O(n^2)$
2. **Drop constants:** If an algorithm is $O(3n)O(3n)O(3n)$, we simplify it to **$O(n)$** .

$\Rightarrow$ Constants (like fixed-time-calculations) Don't affect the Big O Classifications since they don't scale with the input size so , we ignore them

1. Look at loops:

- A **single loop** $\rightarrow O(n)$
 - A **nested loop** $\rightarrow O(n^2)$
 - A **divide-and-conquer approach** (like binary search) $\rightarrow O(\log n)$
-

Most Common Complexities in Interviews

1. $\log(\log n)$ **best**
 2. $\log(\sqrt{n})$, $\log(n)$
 3. $\log^2(n) \Rightarrow$ log squared
 4. $(\sqrt{n})$
 5. $(\sqrt{n}) \log(n)$
 6. n
 7. $n \log(n)$
 8. n^2
 9. n^3
 10. 2^n **worst**
-

What is Space Complexity?

Space complexity = **how much memory an algorithm needs to run.**

It includes:

1. **Fixed part** → constants, program code, and simple variables.
2. **Variable part** → memory that changes with input size (like arrays, linked lists).
3. **Function call stack** → memory used for recursion.

Example:

- Iterative sum of an array → space = $O(1)$
 - Recursive factorial → space = $O(n)$ (because of recursive calls).
-

Stack

- A **stack** is memory used for **function calls**.
- When a function is called, a **stack frame** is created. It contains:
 - Function parameters
 - Local variables
 - Return address (where to go back after function ends)
- When the function ends, its stack frame is removed.
- **Recursion** uses the stack heavily, because each recursive call adds a new frame.

Example:

For `fact(3)` :

- Call `fact(3)` → pushed on stack
- Calls `fact(2)` → pushed
- Calls `fact(1)` → pushed
- Base case returns → stack pops in reverse order

So stack space = recursion depth.

Heap

- The **heap** is memory used for **dynamic allocation** (objects, arrays, linked lists).
- Anything created with `new` (in C++/Java) or dynamically in Python (like lists, dicts) lives in the heap.
- Unlike stack, heap memory is **not automatically freed** when a function ends. You must manage it (in C/C++), or garbage collector does it (Python/Java).

Example:

```
arr = [0]*n
```

This array is stored in the **heap**, not the stack.

4. Stack vs Heap

Feature	Stack	Heap
Purpose	Stores function calls (frames)	Stores dynamically allocated data
Size	Smaller	Larger
Access speed	Very fast (LIFO)	Slower (random access)
Memory managed	Automatically (push/pop)	Manual/GC (new/delete, GC)
Lifetime	Ends when function ends	Until freed / program ends

✓ Summary :

- **Space complexity** = total memory needed.
- **Stack** = function calls + local variables.
- **Heap** = dynamic memory (arrays, objects).